

Task 08: Transformer Reconstruction of IceCube Interaction Positions

Akram Aki

July 20, 2026

1 Introduction

The goal of this task was to reconstruct the two-dimensional neutrino interaction position from IceCube hit sequences. Each event is represented by a variable-length list of measured Cherenkov-light hits. For every hit, the available input features are the detection time and the detector-unit position,

$$(t, x, y),$$

and the regression target is the true interaction position

$$(x_{\text{pos}}, y_{\text{pos}}).$$

The main technical difficulty is that different events contain different numbers of hits. Therefore, the implementation uses a custom batching function, padding, and a padding mask before applying a Transformer encoder.

2 Method

The dataset was loaded from the local IceCube 2D parquet files. The train, validation, and test splits each contained 10001 events. Although the split sizes are identical, the files are distinct; for example, the first target coordinate differs between the three splits. The training events contained between 1 and 68 hits, with a median of 19 hits and a mean of 19.89 hits. Figure 1 shows the hit-count distribution, the one-dimensional target-coordinate distributions, and the two-dimensional target density.

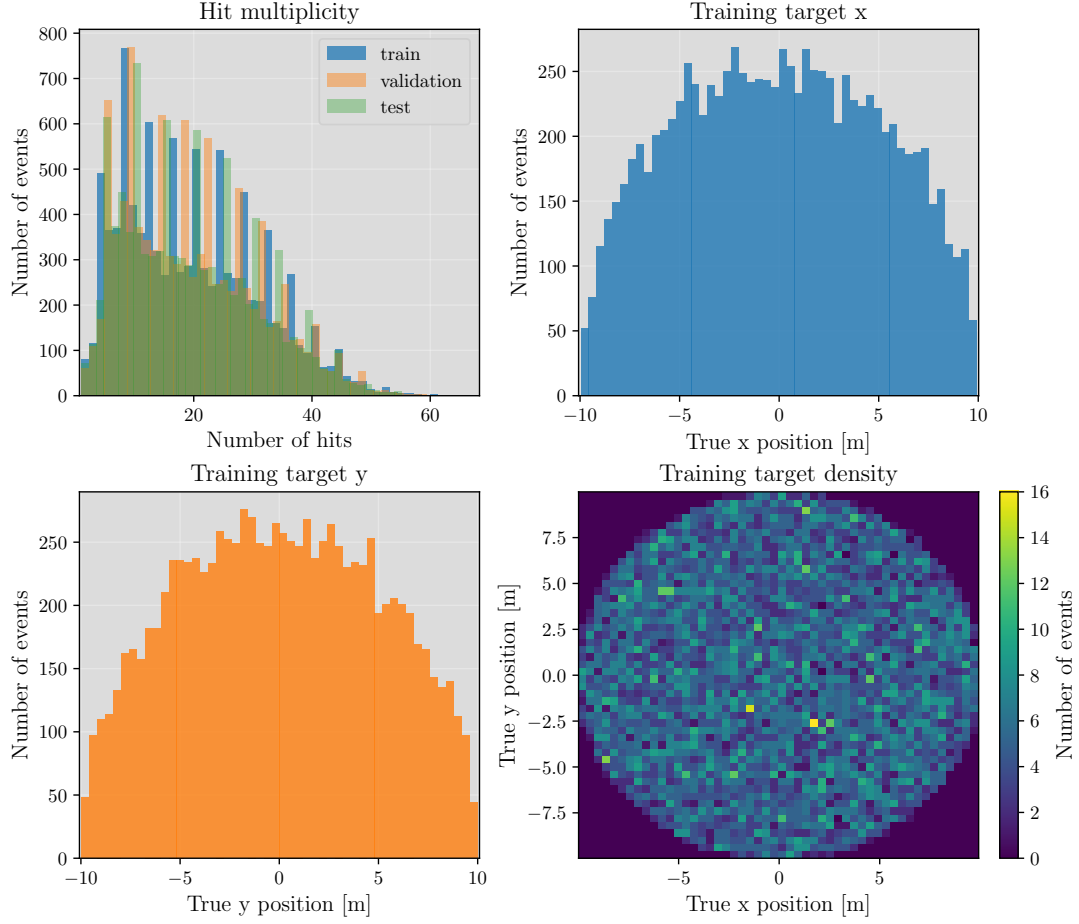


Figure 1: Overview of the IceCube data used in Task 08. The upper-left panel shows the number of hits per event for the three splits. The remaining panels show the training target distributions in x , in y , and in the two-dimensional interaction plane.

All input features and target coordinates were standardized using statistics from the training split only. The same constants were then applied to the validation and test splits. This avoids leaking information from validation or test data into the training procedure. The model was trained in normalized coordinate space, while final metrics were computed after transforming predictions and labels back to meters.

The custom collate function converts each event from the Awkward-array layout $[3, N_{\text{hits}}]$ to the PyTorch sequence layout $[N_{\text{hits}}, 3]$. Within one batch, all hit tensors are concatenated and the original sequence lengths are stored. Inside the model, these lengths are used to split the hits back into events, pad the events to the longest sequence in the batch, and construct a boolean padding mask. The mask is passed to `nn.TransformerEncoder` so that padded positions do not contribute to attention. After the encoder, masked mean pooling averages only over real hits and produces one event-level vector.

The Transformer architecture is summarized in Table 1. It first projects the three hit features to a hidden dimension of 64, applies two Transformer encoder layers with four attention heads, and maps the pooled event representation to the two target coordinates. The model has 71618 trainable parameters.

Quantity	Value
Input features per hit	t, x, y
Targets	$x_{\text{pos}}, y_{\text{pos}}$
Batch size	64
Maximum epochs	50
Early-stopping patience	6
Hidden dimension	64
Attention heads	4
Transformer encoder layers	2
Feed-forward dimension	128
Dropout	0.05
Optimizer	AdamW
Learning rate	10^{-3}
Weight decay	10^{-4}
Loss	Mean squared error
Trainable parameters	71618

Table 1: Transformer and training configuration used for the Task 08 notebook run.

3 Results

Figure 2 shows the training and validation losses in normalized coordinate space. Training was configured for up to 50 epochs, but early stopping stopped the run after epoch 35. The best validation checkpoint was selected at epoch 29, where the validation MSE reached 0.0805. The final test MSE in normalized space was 0.0774.

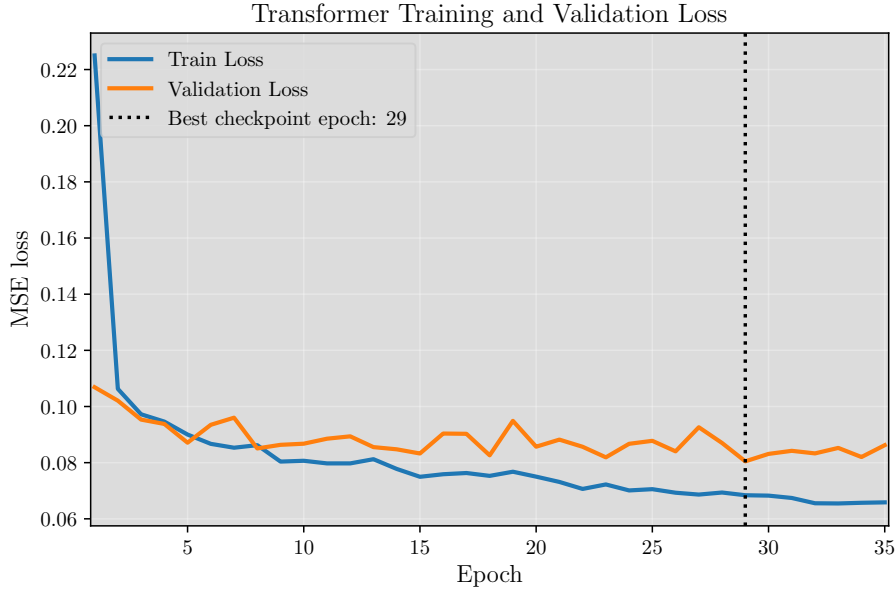


Figure 2: Training and validation loss curves for the Transformer encoder. The vertical line marks the best validation checkpoint, which was used for the final test evaluation.

After denormalizing the predictions and labels, the model reached the metrics shown in Table 2. The mean two-dimensional position error was 1.3492 m, and the median error was 0.8767 m. The R^2 score of 0.9233 indicates that the model explains most of the coordinate variance in the held-out test set.

Metric	Value
Test MSE, normalized space	0.0774
Physical-space MSE	1.9051
MAE in x	0.7802 m
MAE in y	0.9324 m
Mean position error	1.3492 m
Median position error	0.8767 m
R^2 score	0.9233

Table 2: Final test-set metrics after converting the normalized model outputs back to meters.

Figure 3 compares predicted and true coordinates separately. Most points lie close to the perfect-prediction diagonal, but the spread is slightly larger for the y coordinate, consistent with the larger y -MAE in Table 2.

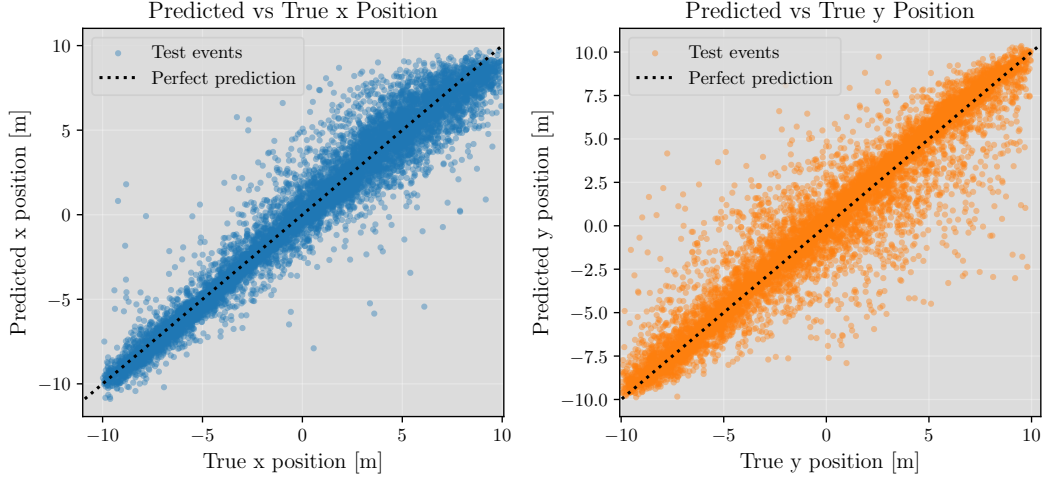


Figure 3: Predicted versus true interaction coordinates on the test set. The diagonal line indicates perfect reconstruction.

Figure 4 shows the full distribution of Euclidean position errors. The distribution is strongly concentrated below a few meters, but it has a tail of harder events with larger reconstruction errors.

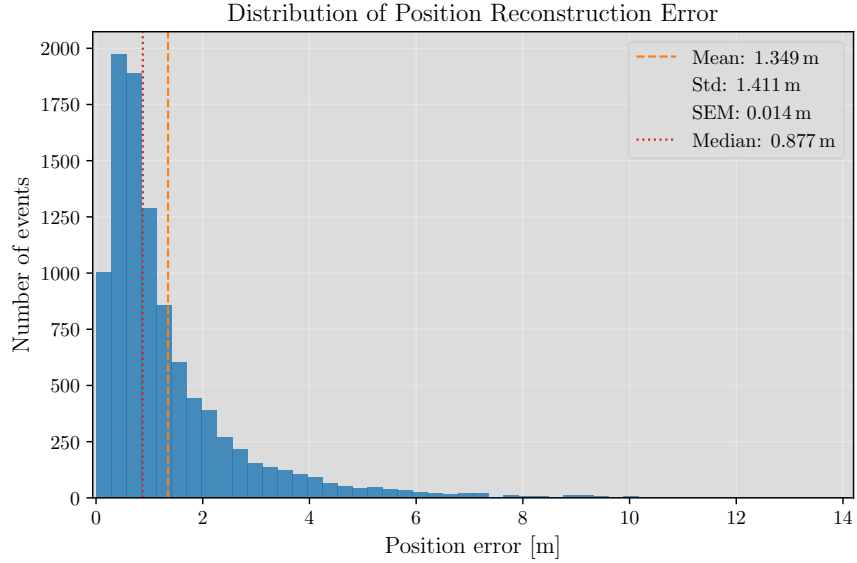


Figure 4: Distribution of two-dimensional position reconstruction errors on the test set.

Finally, Figure 5 checks whether events with more detected hits are easier to reconstruct. The binned trend decreases with increasing hit count, which is physically plausible because more hits provide more timing and geometry information to the Transformer.

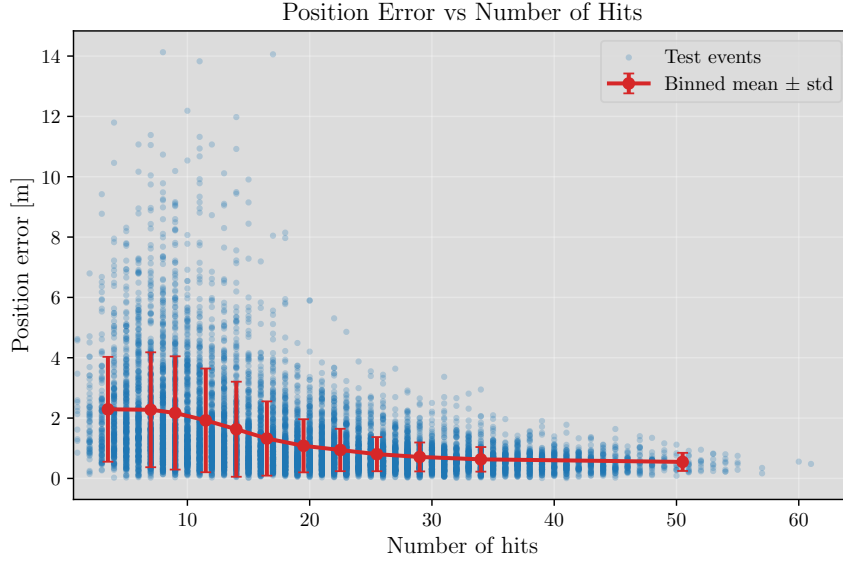


Figure 5: Position error as a function of the number of hits in the test event. The red markers show binned means with standard-deviation bars.

4 Conclusion

The Transformer encoder successfully handled the variable-length IceCube hit sequences using padding masks and masked mean pooling. The model achieved a mean test position error of 1.3492m and a median error below one meter. The error-versus-hit-count diagnostic also shows that events with more detected photons are generally reconstructed more accurately.

Possible improvements include tuning the model size, adding positional or time-order embeddings, using a learning-rate schedule, or comparing masked mean pooling with a learned regression token. One important limitation of this run is the relatively small local training split of 10001 events. The dataset description linked in the task material suggests that a much larger IceCube sample is available, so training on more events would be a natural next step and could improve generalization. It would also be useful to compare the Transformer directly against the previous graph-neural-network reconstruction model on the same split. All code used for data loading, training, evaluation, and plotting is available in the accompanying GitHub repository.